

CSA0381 – DATA STRUCTURES FOR DEPTH FIRST SEARCH

PRACTICAL EXAMINATION PROGRAMS

PREMARUBHA N
192525020

1. MATRIX MULTIPLICATION

main.c	Output
<pre>1 #include <stdio.h> 2 int main() { 3 int a[10][10], b[10][10], c[10][10]; 4 int r1,c1,r2,c2,i,j,k; 5 printf("Enter rows and columns of Matrix A: "); 6 scanf("%d%d",&r1,&c1); 7 printf("Enter rows and columns of Matrix B: "); 8 scanf("%d%d",&r2,&c2); 9 if(c1 != r2) { 10 printf("Multiplication not possible"); 11 return 0; 12 } 13 printf("Enter Matrix A:\n"); 14 for(i=0;i<r1;i++) 15 for(j=0;j<c1;j++) 16 scanf("%d",&a[i][j]); 17 printf("Enter Matrix B:\n"); 18 for(i=0;i<r2;i++) 19 for(j=0;j<c2;j++) 20 scanf("%d",&b[i][j]); 21 for(i=0;i<r1;i++) 22 for(j=0;j<c2;j++) { 23 c[i][j]=0; 24 for(k=0;k<c1;k++) 25 c[i][j]+=a[i][k]*b[k][j]; 26 } 27 }</pre>	<pre>Enter rows and columns of Matrix A: 2 2 Enter rows and columns of Matrix B: 2 2 Enter Matrix A: 1 2 3 4 Enter Matrix B: 5 6 7 8 Result: 19 22 43 50 === Code Execution Successful ===</pre>

2. ODD OR EVEN NUMBER

main.c	Output
<pre>1 #include <stdio.h> 2 int main() { 3 int n; 4 printf("Enter number: "); 5 scanf("%d",&n); 6 7 if(n%2==0) 8 printf("Even"); 9 else 10 printf("Odd"); 11 return 0; 12 }</pre>	<pre>Enter number: 9 Odd === Code Execution Successful ===</pre>

3. FACTORIAL WITHOUT RECURSION

main.c	Output
<pre>1 #include <stdio.h> 2 int main() { 3 int n,i; 4 long long fact=1; 5 printf("Enter number: "); 6 scanf("%d",&n); 7 for(i=1;i<=n;i++) 8 fact*=i; 9 printf("Factorial = %lld",fact); 10 }</pre>	<pre>Enter number: 5 Factorial = 120 === Code Execution Successful ===</pre>

4. FIBONACCI WITHOUT RECURSION

main.c	Output
<pre>1 #include <stdio.h> 2 int main() { 3 int n,a=0,b=1,c,i; 4 printf("Enter terms: "); 5 scanf("%d",&n); 6 for(i=1;i<=n;i++) { 7 printf("%d ",a); 8 c=a+b; 9 a=b; 10 b=c; 11 } 12 }</pre>	<pre>Enter terms: 7 0 1 1 2 3 5 8 === Code Execution Successful ===</pre>

5. FACTORIAL USING RECURSION

main.c	Output
<pre>1 #include <stdio.h> 2 long long fact(int n) 3 { 4 if(n == 0 n == 1) 5 return 1; 6 7 return n * fact(n - 1); 8 } 9 int main() 10 { 11 int n; 12 printf("Enter a number: "); 13 scanf("%d", &n); 14 printf("Factorial = %lld", fact(n)); 15 return 0; 16 }</pre>	<pre>Enter a number: 5 Factorial = 120 === Code Execution Successful ===</pre>

6. FIBONACCI USING RECURSION

main.c	Output
<pre>1 #include <stdio.h> 2 int fib(int n) 3 { 4 if(n <= 1) 5 return n; 6 return fib(n - 1) + fib(n - 2); 7 } 8 int main() 9 { 10 int n, i; 11 printf("Enter number of terms: "); 12 scanf("%d", &n); 13 for(i = 0; i < n; i++) 14 printf("%d ", fib(i)); 15 return 0; 16 }</pre>	<pre>Enter number of terms: 7 0 1 1 2 3 5 8 === Code Execution Successful ===</pre>

7. ARRAY OPERATIONS (INSERT, DELETE, DISPLAY)

main.c	Output
<pre>1 #include <stdio.h> 2 int main() { 3 int a[100], n, i, pos, item; 4 printf("Enter size: "); 5 scanf("%d", &n); 6 for(i=0; i<n; i++) 7 scanf("%d", &a[i]); 8 printf("Insert element and position: "); 9 scanf("%d%d", &item, &pos); 10 for(i=n; i>=pos; i--) 11 a[i]=a[i-1]; 12 a[pos-1]=item; 13 n++; 14 printf("Array after insertion:\n"); 15 for(i=0; i<n; i++) 16 printf("%d ", a[i]); 17 printf("\nDelete position: "); 18 scanf("%d", &pos); 19 20 for(i=pos-1; i<n-1; i++) 21 a[i]=a[i+1]; 22 n--; 23 printf("\nArray after deletion:\n"); 24 for(i=0; i<n; i++) 25 printf("%d ", a[i]); 26 }</pre>	<pre>Enter size: 5 10 20 30 40 50 Insert element and position: 25 3 Array after insertion: 10 20 25 30 40 50 Delete position: 4 Array after deletion: 10 20 25 40 50 === Code Execution Successful ===</pre>

8. LINEAR SEARCH

main.c	Output
<pre>1 #include <stdio.h> 2 int main() 3 { 4 int a[100], n, key, i; 5 printf("Enter number of elements: "); 6 scanf("%d", &n); 7 printf("Enter elements:\n"); 8 for(i = 0; i < n; i++) 9 scanf("%d", &a[i]); 10 printf("Enter element to search: "); 11 scanf("%d", &key); 12 for(i = 0; i < n; i++) 13 { 14 if(a[i] == key) 15 { 16 printf("Element found at position %d", i + 1); 17 return 0; 18 } 19 } 20 printf("Element not found"); 21 return 0; 22 }</pre>	<pre>Enter number of elements: 5 Enter elements: 10 20 30 40 50 Enter element to search: 30 Element found at position 3 === Code Execution Successful ===</pre>

9. BINARY SEARCH

main.c	Output
<pre>1 #include <stdio.h> 2 int main() 3 { 4 int a[100], n, key; 5 int low, high, mid, i; 6 printf("Enter number of elements: "); 7 scanf("%d", &n); 8 printf("Enter sorted elements:\n"); 9 for(i = 0; i < n; i++) 10 scanf("%d", &a[i]); 11 printf("Enter element to search: "); 12 scanf("%d", &key); 13 low = 0; 14 high = n - 1; 15 while(low <= high) 16 { 17 mid = (low + high) / 2; 18 if(a[mid] == key) 19 { 20 printf("Element found at position %d", mid + 1); 21 return 0; 22 } 23 else if(key > a[mid]) 24 low = mid + 1; 25 else 26 high = mid - 1; 27 } 28 printf("Element not found"); 29 }</pre>	<pre>Enter number of elements: 5 Enter sorted elements: 10 20 30 40 50 Enter element to search: 30 Element found at position 3 === Code Execution Successful ===</pre>

10. LINKED LIST OPERATIONS

main.c	Output
<pre>1 #include <stdio.h> 2 #include <stdlib.h> 3 struct node { 4 int data; 5 struct node *next; 6 }; 7 int main() { 8 struct node *head=NULL, *temp, *newnode; 9 int n, i; 10 printf("Enter number of nodes: "); 11 scanf("%d", &n); 12 for(i=0; i<n; i++) { 13 newnode=(struct node*)malloc(sizeof(struct node)); 14 scanf("%d", &newnode->data); 15 newnode->next=NULL; 16 if(head==NULL) 17 head=temp=newnode; 18 else { 19 temp->next=newnode; 20 temp=newnode; 21 } 22 } 23 printf("Linked List:\n"); 24 temp=head; 25 while(temp!=NULL) { 26 printf("%d ", temp->data); 27 temp=temp->next; 28 } 29 }</pre>	<pre>Enter number of nodes: 5 10 20 30 40 50 Linked List: 10 20 30 40 50 === Code Execution Successful ===</pre>

11. STACK OPERATIONS (PUSH, POP, DISPLAY)

main.c	Output
<pre>1 #include <stdio.h> 2 #define MAX 5 3 int stack[MAX]; 4 int top = -1; 5 void push(int item) 6 { 7 if(top == MAX - 1) 8 printf("Stack Overflow\n"); 9 else 10 stack[++top] = item; 11 } 12 void pop() 13 { 14 if(top == -1) 15 printf("Stack Underflow\n"); 16 else 17 printf("Deleted Element = %d\n", stack[top--]); 18 } 19 void display() 20 { 21 int i; 22 if(top == -1) 23 printf("Stack Empty\n"); 24 else 25 { 26 printf("Stack Elements: ");</pre>	<pre>Stack Elements: 30 20 10 Deleted Element = 30 Stack Elements: 20 10 === Code Execution Successful ===</pre>

12. QUEUE OPERATIONS

main.c	Output
<pre>1 #include <stdio.h> 2 #define MAX 10 3 int queue[MAX]; 4 int front=-1, rear=-1; 5 int main() 6 { 7 int n,i,x; 8 printf("Enter number of elements: "); 9 scanf("%d",&n); 10 for(i=0;i<n;i++) 11 { 12 scanf("%d",&x); 13 if(front==-1) 14 front=0; 15 queue[++rear]=x; 16 } 17 printf("Queue Elements: "); 18 for(i=front;i<=rear;i++) 19 printf("%d ",queue[i]); 20 printf("\nDeleted Element = %d",queue[front++]); 21 return 0; 22 }</pre>	<pre>Enter number of elements: 5 1 2 3 5 9 Queue Elements: 1 2 3 5 9 Deleted Element = 1 === Code Execution Successful ===</pre>

13. INSERTION SORT

main.c	Run	Output
<pre>1 #include <stdio.h> 2 int main() 3 { 4 int a[100], n, i, j, temp; 5 printf("Enter size: "); 6 scanf("%d", &n); 7 for(i=0; i<n; i++) 8 scanf("%d", &a[i]); 9 for(i=1; i<n; i++) 10 { 11 temp=a[i]; 12 j=i-1; 13 while(j>=0 && a[j]>temp) 14 { 15 a[j+1]=a[j]; 16 j--; 17 } 18 a[j+1]=temp; 19 } 20 printf("Sorted Array:\n"); 21 for(i=0; i<n; i++) 22 printf("%d ", a[i]); 23 return 0; 24 }</pre>		<pre>Enter size: 5 10 20 50 70 90 Sorted Array: 10 20 50 70 90 === Code Execution Successful ===</pre>

14. SELECTION SORT

main.c	Run	Output
<pre>1 #include <stdio.h> 2 int main() 3 { 4 int a[100], n, i, j, min, temp; 5 printf("Enter number of elements: "); 6 scanf("%d", &n); 7 printf("Enter elements:\n"); 8 for(i = 0; i < n; i++) 9 scanf("%d", &a[i]); 10 for(i = 0; i < n - 1; i++) 11 { 12 min = i; 13 for(j = i + 1; j < n; j++) 14 { 15 if(a[j] < a[min]) 16 min = j; 17 } 18 temp = a[i]; 19 a[i] = a[min]; 20 a[min] = temp; 21 } 22 printf("Sorted Array:\n"); 23 for(i = 0; i < n; i++) 24 printf("%d ", a[i]); 25 return 0; 26 }</pre>		<pre>Enter number of elements: 5 Enter elements: 10 50 75 14 62 Sorted Array: 10 14 50 62 75 === Code Execution Successful ===</pre>

15. BUBBLE SORT

main.c	Output
<pre>1 #include <stdio.h> 2 int main() 3 { 4 int a[50], n, i, j, temp; 5 printf("Enter number of elements: "); 6 scanf("%d",&n); 7 printf("Enter elements:\n"); 8 for(i=0;i<n;i++) 9 scanf("%d",&a[i]); 10 for(i=0;i<n-1;i++) 11 { 12 for(j=0;j<n-i-1;j++) 13 { 14 if(a[j] > a[j+1]) 15 { 16 temp=a[j]; 17 a[j]=a[j+1]; 18 a[j+1]=temp; 19 } 20 } 21 } 22 printf("Sorted Array:\n"); 23 for(i=0;i<n;i++) 24 printf("%d ",a[i]); 25 return 0; 26 }</pre>	<pre>Enter number of elements: 5 Enter elements: 41 85 74 12 96 Sorted Array: 12 41 74 85 96 === Code Execution Successful ===</pre>

16. MERGE SORT

main.c	Output
<pre>1 #include <stdio.h> 2 void merge(int a[], int low, int mid, int high) 3 { 4 int i=low, j=mid+1, k=low, b[50]; 5 while(i<=mid && j<=high) 6 { 7 if(a[i]<a[j]) 8 b[k++]=a[i++]; 9 else 10 b[k++]=a[j++]; 11 } 12 while(i<=mid) 13 b[k++]=a[i++]; 14 while(j<=high) 15 b[k++]=a[j++]; 16 for(i=low; i<=high; i++) 17 a[i]=b[i]; 18 } 19 void mergesort(int a[], int low, int high) 20 { 21 int mid; 22 if(low<high) 23 { 24 mid=(low+high)/2; 25 }</pre>	<pre>Enter size: 5 10 78 65 23 89 Sorted Array: 10 23 65 78 89 === Code Execution Successful ===</pre>

17. QUICK SORT

main.c	Run	Output
<pre>1 #include <stdio.h> 2 void quicksort(int a[], int first, int last) 3 { 4 int i,j,pivot,temp; 5 if(first<last) 6 { 7 pivot=first; 8 i=first; 9 j=last; 10 while(i<j) 11 { 12 while(a[i]<=a[pivot] && i<last) 13 i++; 14 while(a[j]>a[pivot]) 15 j--; 16 if(i<j) 17 { 18 temp=a[i]; 19 a[i]=a[j]; 20 a[j]=temp; 21 } 22 temp=a[pivot]; 23 a[pivot]=a[j]; 24 } 25 } 26 }</pre>		<pre>Enter size: 5 78 42 56 13 89 Sorted Array: 13 42 56 78 89 === Code Execution Successful ===</pre>

18. BST CREATION

main.c	Run	Output
<pre>1 #include <stdio.h> 2 #include <stdlib.h> 3 struct node 4 { 5 int data; 6 struct node *left,*right; 7 }; 8 int main() 9 { 10 struct node *root; 11 root=(struct node*)malloc(sizeof(struct node)); 12 printf("Enter root value: "); 13 scanf("%d",&root->data); 14 root->left=NULL; 15 root->right=NULL; 16 printf("Root Node = %d",root->data); 17 return 0; 18 }</pre>		<pre>Enter root value: 50 Root Node = 50 === Code Execution Successful ===</pre>

19. INORDER TRAVERSAL

```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3 struct node
4 {
5     int data;
6     struct node *left, *right;
7 };
8 struct node* createNode(int data)
9 {
10     struct node *newnode;
11     newnode = (struct node*)malloc(sizeof(struct node));
12     newnode->data = data;
13     newnode->left = newnode->right = NULL;
14     return newnode;
15 }
16 void inorder(struct node *root)
17 {
18     if(root != NULL)
19     {
20         inorder(root->left);
21         printf("%d ", root->data);
22         inorder(root->right);
23     }
24 }
25
26
```

Output

Enter root node: 50
Enter left child: 40
Enter right child: 60
Inorder Traversal: 40 50 60

=== Code Execution Successful ===

20. PREORDER TRAVERSAL OF BST

```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3 struct node
4 {
5     int data;
6     struct node *left, *right;
7 };
8 struct node* createNode(int data)
9 {
10     struct node *newnode;
11     newnode = (struct node*)malloc(sizeof(struct node));
12     newnode->data = data;
13     newnode->left = newnode->right = NULL;
14     return newnode;
15 }
16 void preorder(struct node *root)
17 {
18     if(root != NULL)
19     {
20         printf("%d ", root->data);
21         preorder(root->left);
22         preorder(root->right);
23     }
24 }
25
26 int main()
```

Output

Enter root node: 50
Enter left child: 40
Enter right child: 60
Preorder Traversal: 50 40 60

=== Code Execution Successful ===

21. POSTORDER TRAVERSAL OF BST

main.c	Output
<pre>1 #include <stdio.h> 2 #include <stdlib.h> 3 struct node 4 { 5 int data; 6 struct node *left, *right; 7 }; 8 struct node* createNode(int data) 9 { 10 struct node *newnode; 11 newnode = (struct node*)malloc(sizeof(struct node)); 12 13 newnode->data = data; 14 newnode->left = newnode->right = NULL; 15 16 return newnode; 17 } 18 void postorder(struct node *root) 19 { 20 if(root != NULL) 21 { 22 postorder(root->left); 23 postorder(root->right); 24 printf("%d ", root->data); 25 } 26 }</pre>	<pre>Enter root node: 50 Enter left child: 40 Enter right child: 60 Postorder Traversal: 40 60 50 === Code Execution Successful ===</pre>

22. HEAP SORT

main.c	Output
<pre>1 #include <stdio.h> 2 int main() 3 { 4 int a[50],n,i,j,temp; 5 printf("Enter number of elements: "); 6 scanf("%d",&n); 7 printf("Enter elements:\n"); 8 for(i=0;i<n;i++) 9 scanf("%d",&a[i]); 10 for(i=0;i<n-1;i++) 11 { 12 for(j=i+1;j<n;j++) 13 { 14 if(a[i]>a[j]) 15 { 16 temp=a[i]; 17 a[i]=a[j]; 18 a[j]=temp; 19 } 20 } 21 } 22 printf("Sorted Array:\n"); 23 for(i=0;i<n;i++) 24 printf("%d ",a[i]); 25 return 0; 26 }</pre>	<pre>Enter number of elements: 5 Enter elements: 40 87 56 41 23 Sorted Array: 23 40 41 56 87 === Code Execution Successful ===</pre>

23. BREADTH FIRST SEARCH (BFS)

main.c	Run	Output
<pre>1 #include <stdio.h> 2 int main() 3 { 4 int graph[3][3], visited[3]={0}; 5 int queue[10], front=0, rear=0; 6 int i,j,start; 7 printf("Enter adjacency matrix:\n"); 8 for(i=0;i<3;i++) 9 for(j=0;j<3;j++) 10 scanf("%d",&graph[i][j]); 11 printf("Enter starting vertex (0-2): "); 12 scanf("%d",&start); 13 queue[rear++]=start; 14 visited[start]=1; 15 printf("BFS Traversal: "); 16 while(front<rear) 17 { 18 int node=queue[front++]; 19 printf("%d ",node); 20 for(i=0;i<3;i++) 21 { 22 if(graph[node][i]==1 && !visited[i]) 23 { 24 visited[i]=1; 25 queue[rear++]=i; 26 } 27 } 28 } 29 }</pre>		<pre>Enter adjacency matrix: 0 1 1 1 2 0 0 0 1 Enter starting vertex (0-2): 0 BFS Traversal: 0 1 2 === Code Execution Successful ===</pre>

24. DEPTH FIRST SEARCH (DFS)

main.c	Run	Output
<pre>1 #include <stdio.h> 2 int graph[3][3], visited[3]; 3 void dfs(int v) 4 { 5 int i; 6 visited[v]=1; 7 printf("%d ",v); 8 for(i=0;i<3;i++) 9 { 10 if(graph[v][i]==1 && visited[i]==0) 11 dfs(i); 12 } 13 } 14 int main() 15 { 16 int i,j,start; 17 printf("Enter adjacency matrix:\n"); 18 for(i=0;i<3;i++) 19 for(j=0;j<3;j++) 20 scanf("%d",&graph[i][j]); 21 printf("Enter starting vertex (0-2): "); 22 scanf("%d",&start); 23 printf("DFS Traversal: "); 24 dfs(start); 25 return 0; 26 }</pre>		<pre>Enter adjacency matrix: 0 1 0 1 0 2 0 0 1 Enter starting vertex (0-2): 2 DFS Traversal: 2 === Code Execution Successful ===</pre>

25. DIJKSTRA'S ALGORITHM

main.c	Output
<pre>1 #include <stdio.h> 2 int main() 3 { 4 int cost[3][3]; 5 int dist[3]; 6 int i,j,start; 7 printf("Enter cost matrix:\n"); 8 for(i=0;i<3;i++) 9 for(j=0;j<3;j++) 10 scanf("%d",&cost[i][j]); 11 printf("Enter source vertex: "); 12 scanf("%d",&start); 13 for(i=0;i<3;i++) 14 dist[i]=cost[start][i]; 15 printf("Shortest Distances:\n"); 16 for(i=0;i<3;i++) 17 printf("To %d = %d\n",i,dist[i]); 18 return 0; 19 }</pre>	<pre>Enter cost matrix: 0 4 5 2 4 1 5 0 1 Enter source vertex: 1 Shortest Distances: To 0 = 2 To 1 = 4 To 2 = 1 === Code Execution Successful ===</pre>